The duplicated variable must be equal in all positions of the match clause. Otherwise, you'll get a MatchError.

Now that you have the basics of pattern matching, let's see how it can be used to control the flow of a program.

Use Patterns for Control Flow

Pattern matching goes well beyond assigning variables. Control flow—the branching logic of your application—is performed with pattern matching. Most programming languages rely on if statements for all (or close to all) of an application's control flow. But Elixir offers more choices.

We'll go over four control flow structures: if, case, with, and cond. They all have different purposes and situations where you'll use them, but case statements are the most commonly used.

Let's start with if statements.

If Statements

If statements are so common that it would be a bit strange for Elixir to not have them. Elixir's if statements work largely like you're used to. Use if to quickly check whether a value is truthy or falsy. Here's an example of a simple if statement:

```
$ iex
iex>
if 3 > 5 do
  "if body"
else
  "else body"
end

"else body"
```

Sometimes you'll see if statements on a single line. Elixir allows this single-line syntax for many different keywords:

```
$ iex
iex> if 6 > 5, do: "if body", else: "else body"
"if body"
```

If statements are used to check a single value for truthiness. They can't be used to split a value apart or to compare multiple potential values at the same time. Often, Elixirists will reach for other control flow structures instead of if statements due to these drawbacks. But that doesn't mean if statements are bad! You'll still frequently see and use if statements.